@aman_views

REACT.JS HANDBOOK

1. React Fundamentals & Project Setup

1.1 What is React ?

- React is a JavaScript library for building user interfaces.
- It is maintained by Meta (Facebook) and community.

1.2 Why React ?

- Component based architecture
- Reusable UI components
- Virtual DOM for better performance
- Unidirectional data flow
- Huge ecosystem and community

1.3 React vs Vanilla JavaScript

React	Vanilla JS
Component based	DOM based
Virtual DOM	Real DOM
Reusable components	No component reuse
Fast rendering	Slower rendering
One way data flow	Two way data flow
Easy to maintain	Hard to maintain

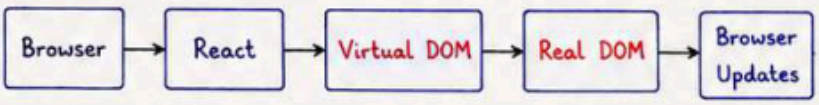
1.4 Virtual DOM vs Real DOM

Real DOM

- Directly updates the real DOM.
- Slower performance.
- Every change causes re-render.

Virtual DOM

- React creates a copy of Real DOM.
- Only changed parts are updated.
- Faster and optimized.



1.5 Creating React Project

Using Vite (Recommended)

1. npm create vite@latest my-app
2. cd my-app
3. npm install
4. npm run dev

Using Create React App

1. npx create-react-app my-app
2. cd my-app
3. npm start

1.6 Folder Structure (Vite)

```
my-app/
├── node_modules/
├── public/
│   └── index.html
├── src/
│   ├── assets/
│   ├── components/
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── .gitignore
├── package.json
└── vite.config.js
```

1.7 Your First React Component

App.jsx

```jsx
function App() {
  return (
    <div>
      <h1> Hello, React! </h1>
      <p> My first component </p>
    </div>
  );
}
```

1.8 Key Takeaway

- React makes it easy to build interactive UIs.
- Components are the building blocks.
- Virtual DOM helps React to be very fast.
- Simple project setup with Vite or CRA.

REACT.JS HANDBOOK

@aman_views

2. JSX & Components

2.1 What is JSX ?

- JSX stands for JavaScript XML.
- It is a syntax extension for JavaScript.
- JSX looks similar to HTML.
- JSX makes it easy to write UI in React.

2.2 JSX Rules

1. JSX must return a single parent element.
2. Always close all tags.
3. Use camelCase for attributes (className, htmlFor).
4. Use className instead of class.
5. Use {} to embed JavaScript expressions.
6. Use style={{}} for inline styling.
7. Comments in JSX use {/* comment */}.

2.3 Expressions in JSX

- You can use any valid JavaScript expression inside {}.
- Examples :

```
<h2>{10 + 5}</h2>
<p>{"Hello React"}</p>
<p>{user.name}</p>
<p>{condition ? "Yes" : "No"}</p>
```

2.4 Functional Component

- A JavaScript function that returns JSX is called a Functional Component.
- It is the simplest way to create a component.

Example :

```
function Welcome() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>This is my first component.</p>
    </div>
  );
}
export default Welcome;
```

2.5 Component Naming Rules

- Component name must start with a capital letter.
- Example : UserCard, ProductList
- Do not use reserved words.
- Use meaningful names.

2.6 Export & Import Components

- Default Export

```
// MyComponent.jsx
export default MyComponent;
```

- Import Default Export

```
import MyComponent from './MyComponent';
```

- Named Export

```
export function MyComponent() {
  return <div>Hi</div>;
}
```

- Import Named Export

```
import { MyComponent } from './MyComponent';
```

2.7 Nested Components

- Components can be used inside other components.
- This helps in building complex UIs.

Example :

```
function Card() {
  return (
    <div className="card">
      <h3>Card Title</h3>
      <p>Card content goes here.</p>
    </div>
  );
}
function App() {
  return (
    <div>
      <h1>My App</h1>
      <Card />
      <Card />
    </div>
  );
}
```

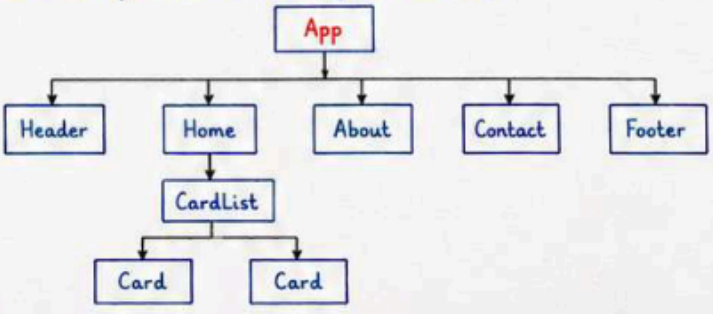
2.8 Reusable Components

- Components can be reused multiple times.
- Helps in clean code and easy maintenance.

Example :

```
function Button(props) {
  return <button onClick={props.onClick}>{props.text}</button>
}
```

2.9 Component Hierarchy (Example)



REACT.JS HANDBOOK

@aman_views

3. Props, State & Event Handling

3.1 What are Props ?

- Props (short for properties) are used to pass data from one component to another.
- Props are read-only (cannot be modified by child component).
- They help in component reusability.

3.2 What is State ?

- State is a built-in object in React.
- State is used to manage dynamic data that changes over time.
- When state changes, component re-renders and UI updates.
- State is private to the component.

3.3 Props vs State

Aspect	Props	State
Definition	Data passed from parent to child	Data managed within the component
Mutability	Immutable (read-only)	Mutable (can change)
Who controls	Parent component	Component itself
Trigger re-render	Yes (when props change)	Yes (when state changes)
Example	<Child name="Aman" />	useState(0)

3.4 Passing Props (Parent → Child)

Parent Component

```
function Parent() {
  return (
    <div>
      <Child name="Aman" age={24} />
    </div>
  );
}
```

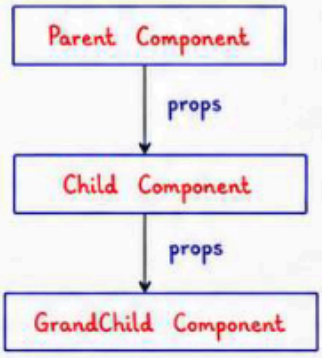
Child Component

```
function Child(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
    </div>
  );
}
```

Important Notes

- Props flow is one-way (Parent → Child)
- Child cannot send props back to parent directly.
- Use callback functions to send data from Child to Parent.

3.5 Parent → Child Data Flow



3.6 useState Hook

- useState is a Hook that lets you add state to functional components.
- It returns an array with two values.
  [state, setState]
  - state → current value
  - setState → function to update the state

Example :

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

3.7 Event Handling

- Events in React are written in camelCase.
- Pass a function as event handler.
- Do not call the function directly.

Example :

```
function Button() {
  const handleClick = () => {
    alert("Button Clicked!");
  };
  return (
    <button onClick={handleClick}>Click Me</button>
  );
}
```

3.8 onChange Example (Form Input)

```
function InputBox() {
  const [text, setText] = useState('');
  return (
    <div>
      <input type="text" value={text}
             onChange={(e) => setText(e.target.value)} />
      <p>You typed: {text}</p>
    </div>
  );
}
```

REACT.JS HANDBOOK

@aman_views

4. React Hooks

- Hooks are special functions that let us "hook" into React features from functional components.
- Hooks were introduced in React 16.8.

4.1 useState

Purpose :
Used to add state to functional components.

Syntax :
```
const [state, setState] = useState(initialValue);
```

Example :
```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

When to use :
When you need local state in a component.

4.2 useEffect

Purpose :
Used to perform side effects in components.
Runs after every render by default.

Syntax :
```
useEffect(() => {
  // side effect code
}, [dependencies]);
```

Example 1 : Run on every render
```
useEffect(() => {
  console.log("Component rendered");
});
```

Example 2 : Run only on mount
```
useEffect(() => {
  console.log("Component mounted");
}, []);
```

Example 3 : Run when dependencies change
```
useEffect(() => {
  console.log("Count changed");
}, [count]);
```

When to use :
- Fetching data
- Subscribing to events
- Manually changing DOM
- Cleanup on unmount

Cleanup Example :
```
useEffect(() => {
  const timer = setTimeout(() => console.log("Hi"), 1000);
  return () => clearTimeout(timer);  // cleanup
}, []);
```

4.3 useRef

Purpose :
- Used to access DOM elements.
- Keeps a mutable value that does not cause re-render.

Syntax :
```
const ref = useRef(initialValue);
```

Example 1 : Access input element
```
const inputRef = useRef();
return <input ref={inputRef} type="text" />;
```

Example 2 : Store a mutable value
```
const countRef = useRef(0);
countRef.current = countRef.current + 1;
```

When to use :
- Access DOM elements
- Store values without re-rendering
- Persist values between renders

4.4 useMemo

Purpose :
Memoizes a computed value.
Recomputes only when dependencies change.

Syntax :
```
const memoizedValue = useMemo(() => computeValue, [deps]);
```

Example :
```
const expensiveValue = useMemo(() => {
  return heavyCalculation(data);
}, [data]);
```

When to use :
When calculation is expensive and depends on some values.

4.5 useCallback

Purpose :
Returns a memoized callback function.
Helps in preventing unnecessary re-renders.

Syntax :
```
const memoizedCallback = useCallback(() => {
  // function code
}, [deps]);
```

Example :
```
const handleClick = useCallback(() => {
  console.log("Button clicked");
}, []);
```

When to use :
When passing functions to child components.

4.6 useContext

Purpose :
Provides a way to share values between components without passing props down manually.

Steps :
1. Create Context
2. Wrap with Provider
3. Use useContext to consume

Example :
```
// Create Context
const ThemeContext = React.createContext();
// In App.js
<ThemeContext.Provider value = "dark">
  <App />
</ThemeContext.Provider>

// In Child Component
const theme = useContext(ThemeContext);
return <div>Current Theme: {theme}</div>;
```

When to use :
When you need global data like theme, user, language in many components.

Quick Summary of Hooks

useState	→	Manage state	useMemo	→	Memoize values
useEffect	→	Handle side effects	useCallback	→	Memoize functions
useRef	→	Access DOM / store mutable values	useContext	→	Share values globally

REACT.JS HANDBOOK

@aman_views

5. Rendering & Forms

5.1 Conditional Rendering

- We can render elements conditionally based on state, props or any condition.

1. if - else

```
function User({isLoggedIn}) {
  if (isLoggedIn) {
    return <h2>Welcome Back!</h2>;
  } else {
    return <h2>Please Login</h2>;
  }
}
```

2. Ternary Operator

```
return (
  <h2>{isLoggedIn ? "Welcome Back!" : "Please Login"}</h2>
);
```

3. && Operator

```
return (
  <div>
    {isLoggedIn && <button onClick={logout}>Logout</button>}
  </div>
);
```

5.2 Rendering Lists

- We use .map() to render lists.
- Each item must have a unique key.

Example :

```
const items = ["Apple", "Banana", "Mango"];
return (
  <ul>
    {items.map((item, index) => (
      <li key={index}>{item}</li>
    ))}
  </ul>
);
```

> **Note**
> Use unique id as key if available. Avoid using index as key in dynamic lists.

5.3 key Attribute

- Helps React identify which items have changed, been added or removed.
- Improves performance and prevents issues.

Good ✓

```
{users.map(user => (
  <div key={user.id}>{user.name}</div>
))}
```

Bad ✗

```
{users.map((user, i) => (
  <div key={i}>{user.name}</div>
))}
```

5.4 Controlled Components

- Form elements whose values are controlled by React state.
- Recommended when we need validation or dynamic input.

Example :

```
import { useState } from 'react';
function Form() {
  const [name, setName] = useState("");
  return (
    <div>
      <input type="text" value={name}
             onChange={(e) => setName(e.target.value)} />
      <p>Your Name: {name}</p>
    </div>
  );
}
```

5.5 Uncontrolled Components

- Form elements whose values are managed by the DOM itself.
- Use when we don't need real-time validation.

Example :

```
import { useRef } from 'react';
function Form() {
  const nameRef = useRef();
  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Name: " + nameRef.current.value);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" ref={nameRef} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

5.6 Handling Form Input

- We can handle multiple inputs using name attribute.

Example :

```
const [form, setForm] = useState({ name: "", email: "" });
const handleChange = (e) => {
  const { name, value } = e.target;
  setForm({ ...form, [name]: value });
};
return (
  <input type="text" name="name" value={form.name}
         onChange={handleChange} />
);
```

5.7 Basic Form Validation

- Validate user input before submitting.
- Show error messages for invalid data.

Example :

```
const [email, setEmail] = useState("");
const [error, setError] = useState("");
const handleSubmit = (e) => {
  e.preventDefault();
  if (!email.includes("@")) {
    setError("Please enter a valid email.");
  return (
    <form onSubmit={handleSubmit}>
      <input type="email" value={email}
             onChange={(e) => setEmail(e.target.value)} />
      {error && <p style={{color: 'red'}}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
  );
```

> **Summary**
> - Conditional rendering helps show/hide UI.
> - Lists make UI dynamic.
> - key improves performance and avoids bugs.
> - Controlled components are best for validation.
> - Uncontrolled components are simple to use.
> - Always validate user input.

REACT.JS HANDBOOK

@aman_views

6. Routing & API Calls

6.1 What is React Router ?

- React Router is a standard library for routing in React applications.
- It helps us build SPA (Single Page Applications).

6.2 Basic Setup

Install :

```
npm install react-router-dom
```

Wrap App.js with BrowserRouter

```
import { BrowserRouter } from 'react-router-dom';
import App from "./App";

function Root() {
  return (
    <BrowserRouter>
      <App />
    </BrowserRouter>
  );
}
export default Root;
```

6.3 Routes, Route & Link

Example :

```
import { Routes, Route, Link } from 'react-router-dom';
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

function App() {
  return (
    <div>
      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link> |
        <Link to="/contact">Contact</Link>
      </nav>
      <hr />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </div>
  );
}
```

Notes

- Link is used for navigation without full page reload.
- Route renders the matched component.
- path="/" is the default (home) route.

6.4 useNavigate Hook

- Used to navigate programmatically (in code).
- Useful after form submit, login, etc.

Example :

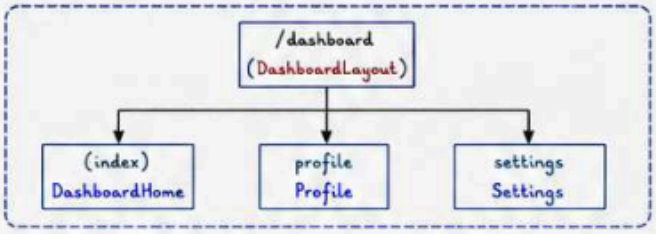
```
import { useNavigate } from 'react-router-dom';
function Login() {
  const navigate = useNavigate();
  const handleLogin = () => {
    // login logic
    navigate('/dashboard');   // redirect
  };
  return <button onClick={handleLogin}>Login</button>;
}
```

6.5 Nested Routes

- Routes inside another route.
- Useful for layouts like Dashboard with Dashboard/Home, Dashboard/Profile etc.

Example :

```
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```



6.6 Fetch API

- Built-in browser API for network requests.
- Returns Promises.

Example :

```
useEffect(() => {
  const fetchData = async () => {
    try {
      const res = await fetch("https://jsonplaceholder.typicode.com/posts");
      const data = await res.json();
      console.log(data);
    } catch (error) {
      console.error("Error:", error);
    }
  };
  fetchData();
}, []);
```

6.7 Axios

- Popular library for making HTTP requests.
- Cleaner syntax than fetch.

Install : npm install axios

Example :

```
import axios from 'axios';
useEffect(() => {
  const fetchData = async () => {
    try {
      const res = await axios.get("https://jsonplaceholder.typicode.com/posts");
      console.log(res.data);
    } catch (error) {
      console.error("Error:", error);
    }
  };
  fetchData();
}, []);
```

6.8 Loading State & Error Handling

Example with Loading :

```
const [loading, setLoading] = useState(true);
useEffect(() => {
  const fetchData = async () => {
    setLoading(true);
    const res = await fetch('...');
    const data = await res.json();
    setData(data);
    setLoading(false);
  };
  fetchData();
}, []);

if (loading) return <p>Loading...</p>;
```

Example with Error Handling :

```
const [error, setError] = useState("");
try {
  const res = await fetch('...');
  if (!res.ok) throw new Error("Failed
     to fetch data");
  const data = await res.json();
  setData(data);
} catch (err) {
  setError(err.message);
}
if (error) return <p style={{color:"red"}}>
  {error}</p>;
```

Quick Summary

React Router	→ Handle navigation in SPA	Fetch / Axios	→ Call APIs
Routes	→ Map paths to components	Loading State	→ Improve user experience
useNavigate	→ Navigate in code	Error Handling	→ Handle failed requests

Tips

- Always show loading while fetching data.
- Handle errors gracefully.
- Use relative paths in nested routes.
- Keep API calls inside useEffect.

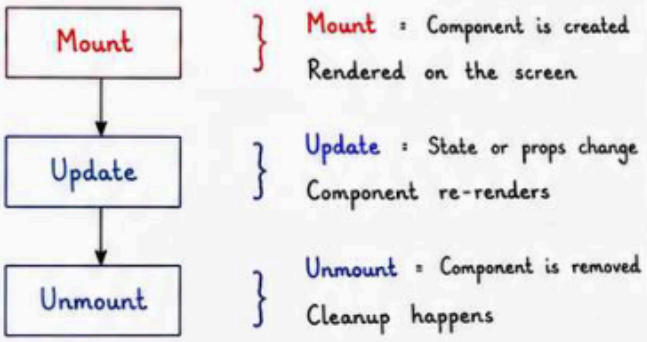
REACT.JS HANDBOOK

@aman_views

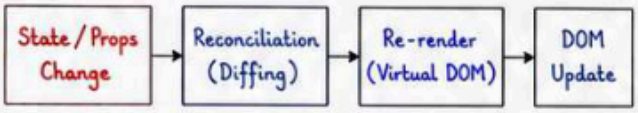
7. Lifecycle & Performance Optimization

7.1 Component Lifecycle

- Every component goes through 3 phases.



7.2 React Rendering Flow



- **Reconciliation :** React compares old and new Virtual DOM.
- **Diffing :** Finds the minimum changes.
- **Re-render :** Updates only what changed.
- **DOM Update :** Applies changes to Real DOM.

7.3 Component Lifecycle (Class Based)

- **Mounting Phase**
  1. constructor()
  2. render()
  3. componentDidMount()
- **Updating Phase**
  1. shouldComponentUpdate()
  2. render()
  3. componentDidUpdate()
- **Unmounting Phase**
  1. componentWillUnmount()

Note

Functional components do not have lifecycle methods. We use `useEffect` Hook instead.

7.4 React.memo

- Higher Order Component.
- Prevents unnecessary re-render.
- Re-renders only when props change.

Example :

```
const Child = React.memo((props) => {
  return <div>{props.name}</div>;
});
```

Use when component is pure and renders same output for same props.

7.5 Lazy Loading & Code Splitting

- Lazy loading loads components on demand.
- Reduces initial bundle size.
- Improves performance.

Example :

```
const About = React.lazy(() => import("./About"));

function App() {
  return (
    <React.Suspense fallback={<p>Loading...</p>}>
      <About />
    </React.Suspense>
  );
}
```

7.6 Performance Best Practices

1. Avoid unnecessary re-renders.
2. Use `React.memo` for pure components.
3. Use `useCallback` for functions.
4. Use `useMemo` for expensive calculations.
5. Code split using `React.lazy`.
6. Keep components small and focused.
7. Avoid inline objects and functions in JSX.

Quick Summary

- Mount → Component appears
- Update → State / props change
- Unmount → Component removed
- React.memo → Prevent re-render
- Lazy Loading → Load on demand
- Best Practices → Better performance

REACT.JS HANDBOOK

@aman_views

8. Project Structure, Testing & Next Steps

8.1 Recommended Project Structure

```
src/
  components/   – Reusable UI components
  pages/        – Page level components
  hooks/        – Custom Hooks
  context/      – Contexts
  services/     – API calls
  utils/        – Helper functions
  App.js        – Root component
  index.js      – Entry point
public/         – Static files
```

8.2 Testing in React

- Testing helps catch bugs early.
- Tools: Jest, React Testing Library
- Write tests for components and user interactions.

Example : Simple Component Test

```
import { render, screen } from
  '@testing-library/react';
import Button from './Button';

test('renders button text', () => {
  render(<Button />);
  const btn = screen.getByText('Click Me');
  expect(btn).toBeInTheDocument();
});
```

8.3 State Management Options

- useState / useReducer – Small to medium apps
- Context API – Share data across components
- Redux Toolkit – Large apps with complex state
- Zustand / Jotai – Simple global state with less boilerplate

8.4 Next Steps for You

1. Build small projects and components.
2. Practice Hooks and state management.
3. Learn React Router by building SPA.
4. Integrate APIs and handle forms.
5. Write tests and follow best practices.

8.5 Common Pitfalls

1. Overusing State
Don't put everything in state. Keep it minimal.

2. Infinite Loops
Check dependencies in useEffect.

3. Unnecessary Re-renders
Use React.memo, useCallback, useMemo.

4. Inline Functions
Avoid creating functions in JSX.

5. Ignoring Keys
Always use unique keys in lists.

Quick Recap

- Learned lifecycle and rendering flow
- Used Hooks for state, refs and effects
- Handled events, forms and validation
- Routing and API calls for real apps
- Optimized performance
- Followed best practices
- Now it's time to build amazing projects!

Keep Learning. Keep Building. You've got this!